

Q. No. 5:

SO: 2 CLO: 6 Total Marks: 6 Obtained Marks: 5

(a) Design a pseudocode to store all the common elements of two sorted arrays of numbers, in new array 'CommonList'.

For example, if List1: 2, 5, 5, 5 and List2: 2, 2, 3, 5, 5, 7, the values stored in CommonList should be 2, 5, 5. (5 points)

(b) What is the maximum number of comparisons your algorithm makes if the lengths of the two given lists are m and n , respectively? (1 point)

Solution:

(a)

Algorithm: FindCommonElements(List1, List2)

Input: Two sorted lists, List1 of size m and List2 of size n

Output: A list of common elements, CommonList

Method:

```
count ← 0
if  $m > n$ 
     $c \leftarrow m$ 
else
     $c \leftarrow n$ 
CmnList[0...c] // c number of elements
for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
        if List1[i] == List2[j] do
            CmnList[count] = List1[i]
            count++
        break
```

return CmnList

(b) Maximum number of comparisons =

$4 \times 6 = 24$ for one element compared with all element in the other list

Q. No. 1: (a) Choose correct option in the following statements.

(i) Which of the following statements is **incorrect** about the definition of algorithm?

- (a) An algorithm is a sequence of unambiguous (i.e. clear) instructions for solving a problem in a finite amount of time.
- (b) An algorithm is a detailed step by step method of solving a problem.
- (c) An algorithm is a procedure to achieve relationship between hardware and software
- (d) An algorithm is a well-defined computational procedure that takes input and produces output.

(ii) Which of the following is **not** a characteristic of an algorithm?

- (a) It accepts zero or more input and produces at least one output
- (b) It is independent of any specific programming language, machine or compiler
- (c) It is understandable to humans and computers
- (d) It stops after finite steps and with correct output

(iii) Which of the following statements is **incorrect** about the space efficiency of algorithms?

- (a) The space efficiency determines total memory requirement of RAM and disk storage to run an algorithm with given input.
- (b) The algorithm code occupies a fixed amount of space, which is dependent on the input size.
- (c) The space requirement consists of the amount of real storage needed to execute the algorithm code and the storage to hold the application data.
- (d) The storage requirement for the application depends on the type of data structure used in the algorithm.

The time efficiency of an algorithm is calculated as:

- (a) Execution time taken by basic operation
- (b) Number of times the basic operation is executed
- (c) Sum of execution time of basic operation and number of times the basic operation is executed
- (d) Product of execution time of basic operation and number of times the basic operation is executed

(v) How "speed efficiency" and "memory efficiency" are related to each other?

- (a) They are normally directly proportional to each other
- (b) They are normally inversely proportional to each other
- (c) Both quantities are required to find the running time of any algorithm
- (d) Both quantities are required to find the space requirement of any algorithm

(vi) Which of the following statements is **incorrect**?

- (a) $\log n \in O(n \log n)$
- (b) $\sqrt{n} \in \Theta(\sqrt{n})$
- (c) $n^3 \in \Omega(n)$
- (d) $3^n \in O(2^n)$

$\checkmark \log n \leq n \log n$
 $\checkmark \sqrt{n} \geq n$
 $\times 3^n \leq 2^n$

$O \rightarrow f(n) \leq g(n)$
 $\Omega \rightarrow f(n) \geq g(n)$
 $\log n \leq$

$\log n$ n $n \log n$ n^2 2^n

Q. No. 2: (a) Solve the following recurrence relation using Backward Substitution method.

$$T(n) = T(n-1) \cdot n, \quad T(0) = 1$$

(3 points)

Solution:

$$(a) \quad T(n) = T(n-1) \cdot n \quad \left\{ \begin{array}{l} T(n-1) = T(n-2) \cdot (n-1) \\ \Rightarrow T(n) = T(n-2) \cdot (n-1) \cdot n \\ \Rightarrow T(n) = T(n-3) \cdot (n-2) \cdot (n-1) \cdot n \\ \vdots \\ T(n) = T(n-k) \cdot (n-k+1) \cdot (n-k+2) \cdot \dots \cdot n \end{array} \right.$$

$$\Rightarrow T(n) = T(n-3) \cdot (n-2) \cdot (n-1) \cdot n$$

$$T(n) = T(n-k) \cdot (n-k+1) \cdot (n-k+2) \cdot \dots \cdot n$$

$$\text{if } n-k=1$$

$$\text{if } n-k=1, T(n-k) = T(0)$$

$$T(n) = T(0) \cdot (n-k+1) \cdot (n-k+2) \cdot \dots \cdot n \Rightarrow T(n) = \Theta(n!)$$

(b) Write recurrence relation for the following pseudocode along with its base case and then solve that recurrence relation using Backward substitution method. (3 points)

Pseudocode:
 $T(n) = 1$ if $n=0$ return 1
 $2T(n-1)$ else return $A(n-1) + A(n-1)$

Solution:

$$T(n) = 2T(n-1) + n, \quad T(0) = 1$$

$$T(n) = 2T(n-1) + n \Rightarrow T(n) = 2[2T(n-2) + (n-1)] + n = 2^2 T(n-2) + 2(n-1) + n$$

$$\Rightarrow T(n) = 4T(n-2) + n-1 + n \Rightarrow T(n) = 4T(n-2) + 2n-1$$

$$\Rightarrow T(n) = 8T(n-3) + n-2 + n-1 + n$$

$$T(n) = 2T(n-k) + (n-k+1) + (n-k+2) + \dots + T(0)$$

$$\text{if } n-k=1, T(n-k) = 1$$

$$\Rightarrow 2 \times 1 + 2 \times 2 + 2 \times 3 + 2 \times 4 + \dots + 2n$$

$$T(n) = \Theta(n^2)$$

Q. No. 4: (a) Consider the following polynomial evaluation algorithm using Brute force approach.

ALGORITHM *BruteForcePolynomialEvaluation*($P[0..n]$, x)
 //Computes the value of polynomial P at a given point x
 //by the "highest to lowest term" brute-force algorithm
 //Input: An array $P[0..n]$ of the coefficients of a polynomial of degree n ,
 // stored from the lowest to the highest and a number x
 //Output: The value of the polynomial at the point x
 $p \leftarrow 0.0$
 for $i \leftarrow n$ downto 0 do
 $power \leftarrow 1$
 for $j \leftarrow 1$ to i do
 $power \leftarrow power * x$
 $p \leftarrow p + P[i] * power$
 return p

Using this algorithm, evaluate the polynomial $P(x) = 6x^2 - 3x - 2$, at $x = 3$ by filling in the following table. (5 points)

i	j	power	P[i]	p
2	1	3	-2	-18
	2	9		
	1	3	-3	-27
0	1	3	6	-9

Final value of p returned:

$p = -9$

Q. No. 3: (a) Consider the following iterative algorithm. Analyze this algorithm to find its running time in terms of Big-O. (3 points)

Algorithm: Sample()

```

for (i ← 1 to n) do
  for (j ← 1 to 2i) do
    k = j
    while (k >= 0) do
      print(k);
      k = k - 1
  
```

Solution:

$$\sum_{i=1}^n \sum_{j=1}^{2i} \sum_{k=0}^{\log n} 2$$

$$\Rightarrow n \sum_{j=1}^{2i} \log n + 1 \Rightarrow 2 \sum_{i=1}^n \log^3 n = 2n \log^3 n = \Theta(n \log^3 n)$$

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^n i = n(n+1)/2$$

$$\sum_{i=1}^n i^2 = (2n^3 + 3n^2 + n)/6$$

$$\sum_{i=0}^n a^i = (a^{n+1} - 1)/(a - 1)$$

(b) Using Master theorem, solve (in detail) the following recurrence relation

$$T(n) = T(2n/3) + 1$$

→ Compare with $n^{\log_b a}$

(3 points)

Solution:

$$a=1, b=3/2, f(n)=1$$

$$f(n)=1$$

$$n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$$

$$\Rightarrow p = n^0 \Rightarrow f(n) = n^{\log_b a}, \text{ Case 2: } \Theta(n^{\log_b a} \log n)$$

$$= \Theta(n^0 \log n) = \Theta(\log n)$$

Q. No. 4: (a) Consider the following polynomial evaluation algorithm using Brute force approach.

ALGORITHM *BruteForcePolynomialEvaluation*($P[0..n]$, x)
 //Computes the value of polynomial P at a given point x
 //by the "highest to lowest term" brute-force algorithm
 //Input: An array $P[0..n]$ of the coefficients of a polynomial of degree n ,
 // stored from the lowest to the highest and a number x
 //Output: The value of the polynomial at the point x
 $p \leftarrow 0.0$
 for $i \leftarrow n$ downto 0 do
 $power \leftarrow 1$
 for $j \leftarrow 1$ to i do
 $power \leftarrow power * x$
 $p \leftarrow p + P[i] * power$
 return p

Using this algorithm, evaluate the polynomial $P(x) = 6x^2 - 3x - 2$, at $x = 3$ by filling in the following table. (5 points)

i	j	power	P[i]	p
2	1	3	-2	-18
	2	9		
1	1	3	-3	-27
0	1	3	6	-9

Final value of p returned:

$P = -9$

(b) Consider the following exhaustive search algorithm for DFS traversal of a graph.

ALGORITHM $DFS(G)$

$count \leftarrow 0$

for each vertex v in V **do**

if v is marked with 0

$dfs(v)$

ALGORITHM $dfs(v)$

$count \leftarrow count + 1;$

mark v with $count$

for each vertex w in V adjacent to v **do**

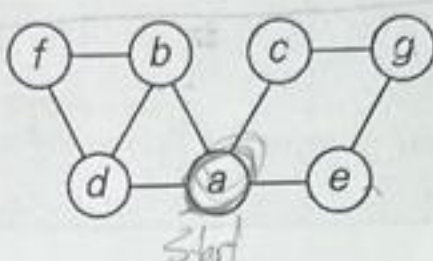
if w is marked with 0

$dfs(w)$

Applying this algorithm on the following graph, find the DFS traversal of the following graph (visit vertices alphabetically).

Show all the recursive calls in the order they are invoked.

(4 points)



Solution:

S. No.	Recursive Calls
1	$DFS(a)$
2	$DFS(b)$
3	$DFS(d)$
4	$DFS(f)$
5	$DFS(c)$
6	$DFS(g)$
7	$DFS(e)$

Traversal Sequence (Using DFS):

a, b, d, f, c, g, e

Pop-off Sequence:

e, g, c, f, d, b, a

MID EXAM CPCS-223

Q No (1)

(3)

c, c, b, d, b, d

Q No (2)

(3)

a) $T(n) = T(n-1) * n$

$= T(n-2) * (n-1) * n$

$= T(n-3) * (n-2) * (n-1) * n$

$= T(n-i) * (n-(i-1)) * (n-(i-2)) * \dots * (n-2) * (n-1) * n$

$= T(n-i) * (n-i+1) * (n-i+2) * \dots * (n-2) * (n-1) * n$

$= T(0) * (0+1) * (0+2) * \dots * (n-1) * n$

$= 1 * 2 * \dots * (n-1) * n = O(n!)$

b) $T(n) = T(n-1) + T(n-1) + 1$ $T(0) = 0$

$= 2T(n-1) + 1$

$= 2[2T(n-2) + 1] + 1 = 2^2 T(n-2) + 2 + 1$

$= 2^2 [2T(n-3) + 1] + 2 + 1$

$= 2^3 T(n-3) + 2^2 + 2 + 1$

$= 2^k T(n-k) + 2^{k-1} + \dots + 2^2 + 2 + 1$

$= 2^k T(n-k) + 2^k - 1$ Let $n=k$

$= 2^n T(0) + 2^n - 1 = 2^n - 1 = O(2^n)$

b) Here

$a=1, b=3/2$ and $f(n)=1$

$\log_a a = \log_{3/2} 1 = 0$

So $n^{\log_a a} = n^0 = 1$

$f(n) = n^{\log_a a}$
So Case 2 applies

$T(n) = O(n^{\log_a a} \log n)$

$= O(\log n)$

Q No (4)

i	j	Power	P[i]	P
2	1	1*3=3	P[2]=6	0+6*9=54
	2	3*3=9		
1	1	1*3=3	P[1]=3	54-9=45
0	-	1	P[0]=-2	45-2=43

Value Returned = 43

b) $dfs(a) \Rightarrow b, c, d, e$

$dfs(b) \Rightarrow a, d, f$

$dfs(d) \Rightarrow a, b, f$

$dfs(f) \Rightarrow b, d$

$dfs(c) \Rightarrow a, g$

$dfs(g) \Rightarrow c, e$

$dfs(e) \Rightarrow a, g$

DFS: a, b, d, f, c, g, e

Pop off: f, d, b, e, g, c, a

Q No (3)

(3)

a) $T(n) = \sum_{i=1}^n \sum_{j=1}^{2i} \sum_{k=0}^j 1 = \sum_{i=1}^n \sum_{j=1}^{2i} (j+1)$

$= \sum_{i=1}^n \left[\sum_{j=1}^{2i} j + \sum_{j=1}^{2i} 1 \right]$

$= \sum_{i=1}^n \left[\frac{2i(2i+1)}{2} + 2i \right] = \sum_{i=1}^n (2i^2 + 3i)$

$= 2 \sum_{i=1}^n i^2 + 3 \sum_{i=1}^n i = \frac{2(2n^3 + 3n^2 + n)}{6} + \frac{3(n)(n+1)}{2} = O(n^3)$

Q No (5) $i=j=k=0$

CommonList[0] = 0

While $i < m$ and $j < n$

if List[i] = List[j]

CommonList[k++] ← List[i]

else CommonList[k++] ← i++

else if List[i] < List[j]

i++

else j++